

sits the question an engineer actually asks of an edited schematic: if I read this drawing as built, what does it do, and is it safe?

We answer that question mechanically. ReSolve recovers the network from the visible SVG alone: wire and pipe endpoints, T-junctions, junction dots, component symbols and their labels, read under standard drafting conventions and with no access to identifiers or metadata. It then solves the recovered network by modified nodal analysis for circuits and by a Newton solve of flows and heads for pipe networks, and compares the solution with the intended design. Because the score depends only on the network, any drawing can be graded the same way, whether it comes from a patch, a re-typed SVG or a human drafter.

Contributions.

- A drawing-grounded verifier for resistor and water-distribution networks. It is invariant to identifiers, element order and uniform scaling, it respects crossing and junction conventions, and it is validated against 10 analytical controls and 10,000 generated drawings, every one of which recovers exactly to its source design (Sec. 3).
- Two physics-verified edit datasets: 10,000 text-guided edits over 2,500 networks for training and evaluation, and a harder evaluation-only tier of 400 edits on networks with up to 16 independent loops. Every gold patch reproduces its target exactly, and every safety verdict is computed from the drawing (Sec. 4).
- A measurement of a frontier model (Astra) under this scoring. It edits every completed task correctly and analyses circuits exactly, but its rare analysis error is silent: it declares an unsafe pipe network safe (Sec. 5).
- A small verified editor. A 1.5B model emitting tree patches, paired with the verifier, is correct on all 1,000 held-out edits and on 99.5% of larger, unseen networks, with exact verdicts. How the patch names elements decides the outcome. Addressing by document position reaches 67.5%, while addressing by SVG identifier reaches 100%.

2. Related work

Vector graphics generation and editing. Learned SVG generators model paths and primitives directly [1] or emit SVG code with a language-model decoder [15]. Instruction-based SVG editing is benchmarked by comparing the edited result with a reference image or markup [14], and scientific-diagram work learns edits from paper revisions [17] or targets structure-faithful generation [10]. These methods treat a drawing as an

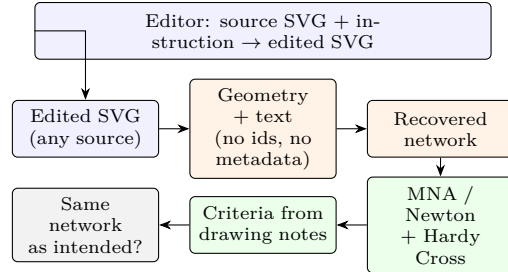


Figure 2. ReSolve. Any edited drawing is read as built: geometry and labels become a network, the network is solved, and the result is checked against the criteria printed on the drawing and compared with the intended design. Identifiers and metadata are never read.

image or a document. We treat an engineering drawing as a specification of a physical network and score it by what that network does.

CAD and engineering design. Generative CAD models produce parametric construction sequences [9, 19], and agentic systems couple language models to simulators for mechanics [13] and finite-element analysis [4, 20]. Benchmarks such as FEABench [12], FEM-Bench [11] and PDEAgent-Bench [5] evaluate the reasoning or code that sets up a simulation. Our verifier instead reads a finished two-dimensional drawing and reconstructs the model it implies, so it can audit the artefact a designer or a model actually delivers.

Network analysis. Modified nodal analysis [6] is the standard formulation for linear circuits. Pipe networks are classically solved by loop corrections [3] or by the gradient method on flows and heads [18] that underlies EPANET [16]; we use Churchill’s friction factor [2], which is continuous from laminar through fully rough flow. None of this is new. What is new is running these solvers on networks recovered from free-form drawings, and using them as the metric for text-guided editing.

3. Drawing-grounded verification

Task. A source drawing S of a network and an instruction u (e.g. “Add $R7 = 10\Omega$ in parallel with $R1$ ”) define a target design D^* . An editor returns a drawing $\hat{S}$. We ask two questions of $\hat{S}$: does it describe D^* (edit correctness), and does the network it describes satisfy the criteria stated on the drawing (verdict)? Both are decided by recovering a network $\mathcal{R}(\hat{S})$ from the drawing and solving it.

3.1. Recovering a network from a drawing

Recovery parses the SVG with transforms applied and rejects scripts, external references, clipping and masking. It then reads only what is visibly drawn.

Connectivity. Stroked straight segments are candidate conductors (for pipe networks, only heavy strokes). A segment endpoint lying in the interior of another segment is a T-junction and splits it; a filled junction dot splits every segment passing through its centre. Two segments that merely cross stay unconnected, which is the drafting convention for both schematics and piping plans. Endpoints closer than $\tau = 10^{-3}$ drawing units are merged. For circuits, all points joined by wire form one electrical node. For pipe networks, nodes are points of degree $\neq 2$, dotted points and points on a reservoir outline, and every chain of segments between two nodes is one pipe. A 90° turn along a chain is an elbow; other bend angles are rejected.

Components. A rectangle whose two short-side mid-points coincide with conductor endpoints is a resistor between the corresponding nodes. A circle is classified by where conductors meet it, not by its size: if a conductor passes through its centre it is a junction dot, and if exactly two conductors end on its rim it is a voltage source, whose positive terminal is the one nearer the “+” mark inside it. Filled rectangles touched by pipe ends are reservoirs. Classifying by incidence rather than radius or stroke width keeps recovery unchanged under uniform scaling of the drawing.

Values and criteria. Labels are parsed with fixed grammars (e.g. R3 4.7 k Ω , P5 320 m $\varnothing$ 150, J2 q=6 L/s $z=10$ m) and assigned one-to-one to the nearest component or pipe, within a reach proportional to the drawing’s own scale (the median resistor body or pipe segment). Design criteria are read from the drawing’s notes: the resistor power rating and supply fuse; the pipe roughness, elbow loss coefficient, minimum pressure head and maximum velocity. A drawing that leaves any component unlabelled, or has two labels competing for one component, is reported as unreadable rather than guessed at.

3.2. Solving the recovered network

Circuits. We use modified nodal analysis [6] with one grounded node per connected part of the network, so a floating fragment carries no current and does not make the system singular. A source whose terminals share a

node, or a loop made only of sources, makes the system singular; such a drawing is reported as ill-posed. The outputs are every branch current, every resistor’s dissipation and every source’s current; the criteria are $P_R \leq P_{\text{rated}}$ for each resistor and $|I_V| \leq I_{\text{fuse}}$ for each source.

Pipe networks. For pipe p from node a to node b with flow Q_p , the head loss is

$$h_p(Q_p) = \left(f(\text{Re}_p, \varepsilon/D_p) \frac{L_p}{D_p} + n_p K_{\text{elbow}} \right) \frac{v_p |v_p|}{2g}, \quad (1)$$

where $v_p = 4Q_p/(\pi D_p^2)$, f is Churchill’s friction factor [2] and n_p is the number of elbows counted from the drawing. We solve jointly for pipe flows and junction heads,

$$h_p(Q_p) = H_a - H_b \quad \forall p, \quad \sum_{\text{in}} Q_p - \sum_{\text{out}} Q_p = q_j \quad \forall j, \quad (2)$$

with reservoir heads fixed, by Newton’s method with backtracking [18], stopping at residuals below 10^{-10} m and 10^{-13} m³/s. Because Churchill’s f tends to $64/\text{Re}$ in laminar flow, h_p has a finite, non-zero slope at $Q_p = 0$ and the Jacobian stays regular when a flow reverses. As an independent check, every network is also solved by Hardy Cross loop corrections [3], with multiple reservoirs joined through a virtual datum node. The criteria are $|v_p| \leq v_{\text{max}}$ for each pipe and $H_j - z_j \geq p_{\text{min}}$ for each junction.

3.3. Scoring an edited drawing

An edit is correct when $\mathcal{R}(\hat{S})$ equals the target network D^* in two senses. The first is topological: we compare, for every node, the multiset of named component terminals it touches, which is a complete invariant of a network with named components, together with every component value and every criterion. The second is physical: the two solutions agree, with currents within 10^{-9} relative plus 1 pA, flows within 10^{-9} m³/s and heads within 10^{-7} m. Markup is irrelevant: a drawing that re-orders elements, renames identifiers or re-formats numbers is credited if it describes the same network, and a drawing that is byte-for-byte plausible but disconnects a wire by 0.01 units is not.

3.4. Validating the verifier

Table 1 lists the analytical controls. They include a balanced and an unbalanced Wheatstone bridge; we initially chose bridge values that happened to balance, which made the “unbalanced” control vacuous and hid a sign error, and we record this as a caution. Beyond the controls, 10,000 independently generated drawings

Control	ReSolve	Reference
series current $V/(R1+R2)$	0.0375	0.0375
parallel current $V/(1/R1+1/R2)$	0.174545	0.174545
balanced Wheatstone bridge current	-1.35×10^{-18}	0
unbalanced bridge vs Thevenin	-0.00307525	-0.00307525
superposition of two sources	0.263636	0.263636
two reservoirs vs bisection (m3/s)	0.0324668	0.0324668
laminar loss vs Hagen-Poiseuille (m)	4.17×10^{-5}	4.17×10^{-5}
Churchill vs Colebrook at $Re=1e6$ (relative gap)	1.00497	1
Newton vs Hardy Cross, two loops + two reservoirs (m3/s)	1.22×10^{-14}	0
mass balance: reservoir supply = demand (m3/s)	1.39×10^{-17}	0

Table 1. Analytical controls for the two solvers. References are closed forms, an independent bisection, or a second algorithm.

(source designs and all their edits, over two seeds) recover to exactly the network they were drawn from, as do all 20,800 drawings in the released datasets. Unit tests confirm four further properties: recovery is unchanged by stripping identifiers, shuffling elements, translating and uniformly scaling the drawing by $\times 0.25$ to $\times 4$; a T-junction joins, while a crossing joins only when dotted; moving only the “+” mark reverses every branch current; and moving one wire endpoint by 0.01 units breaks the connection. Two of these tests found real defects in the first version. A fixed-radius rule for junction dots misclassified dots once the drawing was scaled up, and an absolute current tolerance misjudged an edit whose two sources exactly cancel.

4. Datasets

Resistor networks. A design starts from a grid of 2×3 to 5×5 nodes. Edges are removed at random while the graph stays connected and bridgeless (a bridge would carry no current), until 2–6 independent cycles remain. Each edge becomes a resistor (E6 series, 10Ω – $6.8 \text{ k}\Omega$), a plain wire (20%), or one of one or two DC sources (5–48 V, random polarity). Designs are rejected if a resistor is shorted, if sources form a loop, or if no current flows. Notes state a resistor power rating (0.25–2 W) and a supply fuse (0.1–1 A). Drawings use standard symbols, with junction dots at nodes of degree three or more.

Water-distribution networks. A random spanning tree of a 2×3 to 5×5 grid receives one to three extra pipes to close loops. Some dead ends are pruned, and one or two reservoirs attach at the boundary. Grid points of degree $\neq 2$, plus about a third of the rest, become junctions

	Main (v1)	Hard
Source designs	2,500	100
Edits (train/val/test)	8,000/1,000/1,000	0/0/400
Circuit loops	3 (1–6)	11 (8–16)
Pipe-network loops	2 (1–4)	7 (4–9)
Edited designs safe	5,368	118
Verdict flips	780	22
Edits using set_text	5,722	209
Edits using set_attributes	490	18
Edits using remove_element	2,009	82
Edits using insert_subtree	1,779	91

Table 2. Datasets. Loops are independent cycles of the recovered network (median, range). “Verdict flips” are edits that turn a safe design unsafe or the reverse. All patches reproduce their targets and all drawings recover to their designs.

tions carrying a demand (0–15 L/s) and an elevation (0–30 m). The remaining pass-through points become straight runs or 90° elbows. Pipes have lengths of 100–800 m and nominal diameters of 80–300 mm, and notes state the roughness, elbow loss coefficient ($K = 0.9$), minimum pressure head (10–20 m) and maximum velocity (1.5–2.5 m/s).

Edits. Each design receives four instructions. For circuits these are a resistor value change, a source voltage change, a polarity reversal (two-source designs) or opening a branch (one-source designs, provided the network stays solvable), and adding a resistor in parallel with an existing one, drawn as a tapped branch. For pipe networks they are a diameter change, a demand change, removing a pipe (provided every junction stays supplied and no reservoir is cut off), and adding a pipe between adjacent junctions (or, if none exist, a length change). Target drawings are rendered from the edited design. The gold edit is a tree patch derived between source and target SVG, applied with a validating executor and required to reproduce the target exactly. A row is kept only if both drawings recover to their designs and, for pipes, Newton and Hardy Cross agree to $10^{-9} \text{ m}^3/\text{s}$.

Statistics. Table 2 summarises both tiers. The hard tier uses 4×5 to 5×6 grids with 8–16 independent loops for circuits and four to eight added pipes for water networks. It is evaluation-only: no model is trained on networks of that size. Splits are by source design, so the four edits of one drawing never cross splits.

5. Experiments

Frontier model. We evaluate Astra (gpt-6-astra) through the Chat Completions API at medium reason-

ing effort, with a 32,000-token completion cap, no tools, store=false and one sample per task. Astra works in its natural mode: it receives the drafting conventions, the physical model (fluid properties, friction law, loss conventions) and the criteria in the prompt, and it returns the complete edited SVG followed by a JSON analysis stating the verdict, the violating components and the governing quantities. The edited SVG is scored by ReSolve exactly as any other drawing (Sec. 3), and the analysis is compared with the solver’s. Tasks are the first ten test edits by identifier for every (family, edit type) pair of the main tier (100 tasks), and the first four of the hard tier (40 tasks). The API account ran out of credit during the run. All 79 main-tier and 23 hard-tier tasks that completed are reported. Every circuit task completed. The missing tasks are all pipe networks, which failed with “no credits remaining” (26) or a dropped connection (12), and they are excluded rather than counted as failures.

Our editor. ReSolve pairs the verifier with a small editor: Qwen2.5-Coder-Instruct [8] at 1.5B and 7B parameters, fine-tuned with LoRA [7] (rank 16 on the attention projections) for two epochs on the 10,000-edit training split to map source SVG + instruction to a tree patch. The patch is applied by a validating executor, and the resulting drawing is recovered and solved, so the verdict is computed, not generated. Decoding is greedy. The hard tier is never seen in training. We compare two ways for a patch to name the elements it changes. Positional addressing uses the executor’s native node identifiers, where n77 is the 77th element in document order. Id addressing uses the element’s own SVG identifier (#R5-label) and writes insertion at the end of a group as “end”. It is translated to positional form before execution, so the two differ only in what the model must write.

Metrics. Edit: the edited drawing recovers to the target network. Verdict: the stated pass/fail matches the solver’s. Violations: the set of violating components matches exactly. All: all three at once. For systems whose verdict is computed by ReSolve, the verdict is taken from the solve of the edited drawing, whether or not the edit was correct.

5.1. What Astra gets right

Astra is an accurate editor of these drawings. Every one of its completed edits recovers to the target network, including parallel branches drawn as tapped sub-circuits and pipes added between junctions, and it never disconnected a wire or broke the drafting conventions. Its circuit analysis is exact on every completed

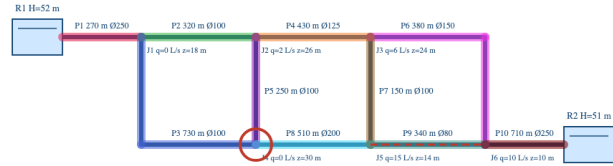


Figure 3. A confident wrong verdict. “Change the length of P9 to 340 m” (dashed) in a two-reservoir network with four loops. Astra’s edit is correct, and it reports the minimum pressure head at J4 (circled) as 20.04 m and the design as passing. Re-solving the drawing gives 19.23 m, below the 20 m minimum stated on the drawing. Several of its pipe velocities are off by up to 67 %.

task, including hard-tier networks with up to 16 independent loops. We did not find the failure boundary we expected for linear networks: at these sizes, unaided nodal analysis is within reach. Each answer costs \$0.20 on the main tier and \$0.38 on the hard tier on average, and takes a median of 44 s and 101 s respectively, with no truncated responses.

5.2. Where it fails: silently

The only substantive error among Astra’s completed answers is in a nonlinear pipe network, and it is not flagged (Fig. 3). The edit is right, but the analysis declares a failing design safe: it reports 20.04 m of pressure head where the drawing yields 19.23 m against a 20 m minimum, and its flows are wrong throughout the loop that contains the edited pipe. Nothing in the answer distinguishes it from the correct ones: it finished normally, used a similar reasoning budget, and presented its numbers with the same precision. This is the failure mode a verifier exists for. It is rare (1 of 79 main-tier answers), it concentrates where the physics is nonlinear and the design sits near a limit, and it cannot be detected from the answer itself. Replacing Astra’s analysis with the solve of its own drawing removes it (Tab. 3, second row).

5.3. The small editor

Table 4 and Fig. 4 show the editor’s results. With id addressing, the 1.5B editor is correct on every one of the 1,000 main-tier test edits, and on 99.5 % of the hard tier, whose networks are larger than anything in training. On the tasks Astra completed it matches Astra’s edits exactly (Tab. 3). Because its verdict is computed from its own drawing, its verdicts are correct whenever its edits are. A 7B editor trained identically reaches the same totals (1,000 of 1,000 and 99.5 %) with different misses (below). At this task size, model scale buys nothing that the representation does not already provide.

System	Edit	Verdict	Violations	All
Main tier (same completed tasks for every system)				
Circuits				
Astra (own analysis)	100 (50/50)	100 (50/50)	100 (50/50)	100 (50/50)
Astra drawing + ReSolve verdict	100 (50/50)	100 (50/50)	100 (50/50)	100 (50/50)
Qwen2.5-Coder 1.5B, untrained	0.0 (0/50)	0.0 (0/50)	0.0 (0/50)	0.0 (0/50)
ReSolve editor 1.5B	100 (50/50)	100 (50/50)	100 (50/50)	100 (50/50)
ReSolve editor 7B	100 (50/50)	100 (50/50)	100 (50/50)	100 (50/50)
Pipes				
Astra (own analysis)	100 (29/29)	96.6 (28/29)	96.6 (28/29)	96.6 (28/29)
Astra drawing + ReSolve verdict	100 (29/29)	100 (29/29)	100 (29/29)	100 (29/29)
Qwen2.5-Coder 1.5B, untrained	0.0 (0/29)	0.0 (0/29)	0.0 (0/29)	0.0 (0/29)
ReSolve editor 1.5B	100 (29/29)	100 (29/29)	100 (29/29)	100 (29/29)
ReSolve editor 7B	100 (29/29)	100 (29/29)	100 (29/29)	100 (29/29)
Hard tier (8–16 loops for circuits; unseen in training)				
Circuits				
Astra (own analysis)	100 (20/20)	100 (20/20)	100 (20/20)	100 (20/20)
Astra drawing + ReSolve verdict	100 (20/20)	100 (20/20)	100 (20/20)	100 (20/20)
ReSolve editor 1.5B	100 (20/20)	100 (20/20)	100 (20/20)	100 (20/20)
ReSolve editor 7B	100 (20/20)	100 (20/20)	100 (20/20)	100 (20/20)
Pipes				
Astra (own analysis)	100 (3/3)	100 (3/3)	100 (3/3)	100 (3/3)
Astra drawing + ReSolve verdict	100 (3/3)	100 (3/3)	100 (3/3)	100 (3/3)
ReSolve editor 1.5B	100 (3/3)	100 (3/3)	100 (3/3)	100 (3/3)
ReSolve editor 7B	100 (3/3)	100 (3/3)	100 (3/3)	100 (3/3)

Table 3. Edit and verdict accuracy, % (count), on the tasks Astra completed. “Astra drawing + ReSolve verdict” keeps Astra’s edited SVG but replaces its analysis with the solve of that drawing.

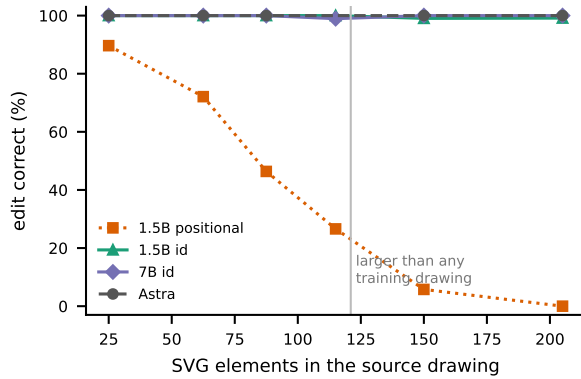


Figure 4. Edit accuracy against drawing size, pooled over both tiers and binned by the number of SVG elements in the source drawing (bins with at least five edits). The grey line marks the largest drawing in the training split.

Addressing matters more than scale. The same model and data with positional addressing reaches only 67.5 % (circuits 43.6 %, pipes 91.4 %) and collapses to 16.5 % on the hard tier. Accuracy falls steadily with the number of elements in the drawing (Fig. 4): to name a label as `n77` the model must count through the document. In pipe drawings a pipe’s label position follows from its number and the pipe count, while in circuits

it depends on how many sources precede it (each contributes three text elements), and circuits are where positional addressing fails. Naming elements by their SVG identifier removes the counting and costs nothing at inference.

Remaining errors and markup. The 1.5B editor’s two hard-tier misses both add a parallel branch to the largest circuits: the model misplaces the branch’s vertical coordinates by 8–30 units, so it no longer taps the intended wires, on rows lower in the drawing than any seen in training. The 7B editor gets every hard circuit right. Its two misses add a pipe to the largest water networks, drawing the pipe exactly but placing its label 91–100 units below it, which leaves the new pipe unlabelled. The verifier rejects all four drawings, the first two because the recovered network differs and the last two because they cannot be read. In the other direction, 22 of the 1.5B editor’s main-tier edits (6 of the 7B’s) are correct networks but not byte-identical to the gold drawing. For the 1.5B editor, 18 leave a junction dot that the gold drawing removes after opening a branch, and 4 place an added pipe’s label 1–12 units from the gold position. A markup metric would count these as failures. Scoring by the recovered network credits them.

Editor	Circuits	Pipes	All
1.5B pos., main	43.6 (218/500)	91.4 (457/500)	67.5 (675/1000)
1.5B pos., hard	0.5 (1/200)	32.5 (65/200)	16.5 (66/400)
1.5B id, main	100 (500/500)	100 (500/500)	100 (1000/1000)
1.5B id, hard	99.0 (198/200)	100 (200/200)	99.5 (398/400)
7B id, main	100 (500/500)	100 (500/500)	100 (1000/1000)
7B id, hard	100 (200/200)	99.0 (198/200)	99.5 (398/400)

Table 4. Our editor on the full test sets: edit correctness by family and end-to-end accuracy (“pos.” = positional, “id” = id addressing) on all 1,000 main-tier and 400 hard-tier edits, % (count).

5.4. Cost of a verified edit

An Astra answer costs \$0.20 (main tier) to \$0.38 (hard tier) at list price and takes a median of 44s to 101s. Our 1.5B editor trains in 36 minutes on one RTX PRO 6000 GPU and then needs 1.11s of GPU time per edit when batched sixteen at a time. The id-addressed run shared the GPU with another user’s jobs, which stretched its training to 90 minutes and inference to 1.53s per edit. The 7B editor trains in 111 minutes on one H100 NVL and needs 1.30s per edit. The verifier needs a median of 4ms (main) and 13ms (hard) of laptop CPU time per drawing to recover, solve, check and compare it. It is cheap enough to run on every edit from any source, including Astra’s.

6. Limitations and conclusion

Limitations. The networks are procedural: rectilinear layouts, a fixed symbol vocabulary and label grammar, steady DC circuits with ideal sources, and steady incompressible pipe flow with elbow losses only. Recovery is strict by design. A drawing that uses other symbols, draws a resistor as a zig-zag, or places labels far from their components is reported as unreadable, not interpreted. The frontier-model study covers one model at one reasoning effort with one sample per task, and its pipe-network arm is incomplete because the API account ran out of credit, so the hard pipe tier in particular is under-measured. The single wrong verdict is a demonstrated failure mode, not an estimated rate. The numerical tolerance for Astra’s reported velocities gained an absolute floor of 0.01m/s after we saw that three-decimal rounding of near-zero velocities was being scored as error. Verdicts and edits are unaffected by this change.

Conclusion. An edited engineering drawing can be graded by reading it as built and solving what it describes. This removes the dependence on markup that makes current SVG-editing metrics fragile, and it turns “is this edit safe?” into a computation instead of a

claim. Under that grading a frontier model is a strong editor and usually a strong analyst, but its rare analysis errors are silent. Pairing any editor with a drawing-grounded solve makes the verdict exact whenever the edit is right. A 1.5B editor that names elements by identifier, paired with the verifier, matches the frontier model’s edits on the tasks both attempted. It is correct on all 1,000 main-tier and 99.5% of unseen hard-tier edits, its verdicts are exact by construction, and it runs locally in about a second per edit.

References

- [1] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. DeepSVG: A hierarchical generative network for vector graphics animation. In *Advances in Neural Information Processing Systems*, 2020. 2
- [2] Stuart W. Churchill. Friction-factor equation spans all fluid-flow regimes. *Chemical Engineering*, 84(24): 91–92, 1977. 2, 3
- [3] Hardy Cross. Analysis of flow in networks of conduits or conductors. *University of Illinois Engineering Experiment Station Bulletin*, (286), 1936. 2, 3
- [4] Rushikesh Deotale, Adithya Srinivasan, Yuan Tian, Tianyi Zhang, Pavlos Vlachos, and Hector Gomez. All-fem: Agentic large language models fine-tuned for finite element methods, 2026. 2
- [5] Zhen Hang, Yushan Yashengjiang, Junhui Li, Huan-shuo Dong, Yang Wei, Zhezheng Hao, Jiangtao Ma, Songlin Bai, Haozhong Kai, Xihang Yue, Gangzong Si, Dongming Jiang, Chao Yao, Zhanhua Hu, Jiangqing Zhang, Pengwei Liu, Yaomin Shen, Xingyu Ren, Lei Liu, Zikang Xu, Han Li, Qingsong Yao, Hande Dong, and Hong Wang. Pdeagent-bench: A multi-metric, multi-library benchmark for pde solver generation, 2026. 1, 2
- [6] Chung-Wen Ho, Albert E. Ruehli, and Pierce A. Brennan. The modified nodal approach to network analysis. *IEEE Transactions on Circuits and Systems*, 22 (6):504–509, 1975. 2, 3
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. 5
- [8] Binyuan Hui et al. Qwen2.5-coder technical report, 2024. 5
- [9] Mohammad Sadil Khan et al. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In *Advances in Neural Information Processing Systems*, 2024. 2
- [10] Yuxuan Luo, Peng Zhang, Xinjie Zhang, Xun Guo, Zhouhui Lian, and Yan Lu. Sciforma: Structure-faithful generation of scientific diagrams, 2026. 2
- [11] Saeed Mohammadzadeh, Erfan Hamdi, Joel Shor, and Emma Lejeune. Fem-bench: A structured scientific

- reasoning benchmark for evaluating code-generating
llms, 2025. [1](#), [2](#)
- [12] Nayantara Mudur, Hao Cui, Subhashini Venugopalan,
Paul Raccuglia, Michael P. Brenner, and Peter Nor-
gaard. Feabench: Evaluating language models on mul-
tiphysics reasoning ability, 2025. [1](#), [2](#)
- [13] Bo Ni and Markus J. Buehler. Mechagents: Large
language model multi-agent collaborations can solve
mechanics problems, generate new data, and integrate
knowledge, 2023. [2](#)
- [14] Kunato Nishina and Yusuke Matsui. Svgeditbench v2:
A benchmark for instruction-based svg editing, 2025.
[1](#), [2](#)
- [15] Juan A. Rodriguez et al. StarVector: Generating scal-
able vector graphics code from images and text. In
Proceedings of the IEEE/CVF Conference on Com-
puter Vision and Pattern Recognition (CVPR), 2025.
[2](#)
- [16] Lewis A. Rossman. EPANET 2 users manual. Tech-
nical Report EPA/600/R-00/057, U.S. Environmental
Protection Agency, 2000. [2](#)
- [17] Yasheng Sun, Zezi Zeng, Yifan Yang, Chong Luo,
Wenyi Wang, Ziwei Liu, and Jürgen Schmidhuber.
Scidiagramedit: Learning to edit scientific diagrams
from paper revisions, 2026. [1](#), [2](#)
- [18] Ezio Todini and S. Pilati. A gradient algorithm for the
analysis of pipe networks. In Computer Applications
in Water Supply, Vol. 1, pages 1–20. Research Studies
Press, 1988. [2](#), [3](#)
- [19] Rundi Wu, Chang Xiao, and Changxi Zheng. Deep-
CAD: A deep generative network for computer-aided
design models. In Proceedings of the IEEE/CVF In-
ternational Conference on Computer Vision (ICCV),
2021. [2](#)
- [20] Jiachen Zhang, Junyi Lao, Chenghao Liu, Siyuan Liu,
Shixin Wu, Linsen Zhang, Boyu Wang, and Songfang
Huang. Vfeagent: A multimodal agent framework for
end-to-end automated finite element analysis, 2026. [2](#)

A. Supplementary material

A.1. Prompt given to Astra

System message: You are an engineering drafting and analysis assistant. Use your own reasoning; no external tools are provided. Do not claim you ran a solver.

User message for a circuit task (the source SVG follows verbatim):

Instruction: Set R3 to 150 Ω .

Drawing conventions: straight lines are ideal wires. A wire that ends on another wire (a T) is connected to it; a filled dot also marks a connection; wires that cross without a dot are NOT connected. Rectangles are resistors and circles are ideal DC voltage sources; the '+' inside a source marks its positive terminal. Labels give component names and values.

Criteria (from the drawing notes): every resistor's dissipated power must not exceed the stated rating, and the magnitude of every source's current must not exceed the stated supply fuse rating.

Tasks:

1. Apply the instruction to the drawing, changing nothing else, and return the complete edited SVG.
2. Analyse the EDITED design and report whether it meets every criterion.

Answer with exactly two fenced blocks and nothing else: first "svg" containing the full edited SVG, then "json" containing {"verdict": "pass" or "fail", "violations": [names of every resistor or source that violates a criterion], "source_currents_A": {"V1": current delivered out of its + terminal, ...}, "max_resistor_power_W": {"name": "R?", "value": watts}}

Source SVG: [complete SVG]

For pipe networks the conventions paragraph and JSON schema are replaced by:

Drawing conventions: heavy lines are pipes; filled dots are junctions. A pipe runs between junctions or reservoirs and may turn through 90-degree elbows; each elbow adds the minor-loss coefficient K given in the notes. 'P# L m $\varnothing$ D' gives a pipe's length (m) and internal diameter (mm); 'J# q=.. L/s z=.. m' gives the demand withdrawn at a junction and its elevation; 'R# H=.. m' is a reservoir's fixed total head.

Analysis model: steady incompressible water, density 998 kg/m³, dynamic viscosity 1.002e-3 Pa s, g = 9.80665 m/s²; Darcy-Weisbach head loss with the Churchill (1977) friction factor and the pipe roughness in the notes; tee and junction losses neglected; pressure head at a junction = hydraulic head minus elevation.

Criteria (from the drawing notes): every pipe velocity must not exceed the maximum velocity, and every junction pressure head must be at least the minimum pressure head.

JSON: {"verdict": "pass" or "fail", "violations": [names of every pipe or junction that violates a criterion], "pipe_velocities_m_s": {"P1": speed, ...}, "min_pressure_head_m": {"name": "J?", "value": metres}}

A.2. Results by edit type

A.3. Training stability

The first 7B training run, on an H100 NVL, diverged: the loss rose from 0.65 at step 20 to 3.0×10^4 at step 30 with a non-finite gradient norm, and the resulting adapter emits only "!" characters. Two 50-step re-runs with identical code, settings and data order isolate the cause to PyTorch's cuDNN kernel for scaled dot-product attention. With it enabled the gradient norm again became non-finite at step 30. With it disabled all 50 steps were finite and the loss fell from 0.55 to 0.026. The reported 7B editor was retrained with that kernel disabled. The 1.5B editors, trained on an RTX PRO 6000, never used it. The diverged run is

retained, marked void, and excluded from every table, and the trainer now stops at the first non-finite loss or gradient.

601

602

603

Edit type	Astra	1.5B pos.	1.5B id	7B id
Circuits: component value	100 (10/10)	31.2 (39/125)	100 (125/125)	100 (125/125)
Circuits: open branch	100 (10/10)	41.6 (32/77)	100 (77/77)	100 (77/77)
Circuits: parallel branch	100 (10/10)	53.6 (67/125)	100 (125/125)	100 (125/125)
Circuits: polarity	100 (10/10)	29.2 (14/48)	100 (48/48)	100 (48/48)
Circuits: source value	100 (10/10)	52.8 (66/125)	100 (125/125)	100 (125/125)
Pipes: add pipe	100 (1/1)	100 (60/60)	100 (60/60)	100 (60/60)
Pipes: junction demand	100 (7/7)	66.4 (83/125)	100 (125/125)	100 (125/125)
Pipes: pipe diameter	100 (8/8)	99.2 (124/125)	100 (125/125)	100 (125/125)
Pipes: pipe length	83.3 (5/6)	100 (65/65)	100 (65/65)	100 (65/65)
Pipes: remove pipe	100 (7/7)	100 (125/125)	100 (125/125)	100 (125/125)

Table 5. End-to-end accuracy (edit, verdict and violations all correct), % (count). Astra on its completed main-tier tasks; ReSolve editors (positional or id addressing) on the full 1,000-edit main test set.